

result

October 7, 2026

Benchmarking de Algoritmos de Reglas de Asociación en Cesta de la Compra

Dataset: *The Bread Basket Dataset* (Panadería ‘The Bread Basket’, Edimburgo) **Fuente original:** [Kaggle Dataset](#)

0.0.1 Objetivos del Proyecto y Propósito del Benchmarking (Diapositivas 38-46):

1. Exploración inicial y preprocesamiento:

- Análisis de distribución temporal (horas, períodos del día, fines de semana vs días laborales).
- Distribución de frecuencias de productos y tamaño de cesta de la compra.
- Modificación mínima y justificada del dataset: limpieza de espacios en blanco, exclusión fundamentada de registros no comerciales/administrativos y deduplicación intrasensorial.

2. Implementación y comparación de paradigmas algorítmicos:

- **Apriori (mlxtend y efficient-apriori):** Búsqueda en anchura por niveles (BFS) con generación de candidatos y poda antimonótona.
- **FP-Growth (mlxtend):** Enfoque *divide and conquer* basado en árbol de patrones frecuentes (FP-Tree) sin generación de candidatos.
- **ECLAT:** Enfoque vertical basado en listas de identificadores de transacción (TID-sets) e intersección de conjuntos.

3. Benchmarking computacional multidimensional:

- Evaluación del tiempo de ejecución (CPU time con alta resolución) y consumo de memoria pico (`tracemalloc`).
- Sensibilidad ante variaciones del umbral de soporte mínimo (*min_support*).
- Comparación de la rejilla 2D combinada de (*soporte* $\times$ *confianza*) evaluando volumen total de reglas y **reglas útiles** (*Lift* $>$ 1.0).

4. Evaluación multivariante de las 10 métricas de asociación (Diapositivas 44-46):

- Antecedente, Consecuente, Soporte del Antecedente, Soporte del Consecuente, Soporte, Confianza, Lift, Leverage, Convicción, Cobertura y Métrica de Zhang.

5. Interpretación comercial y conclusiones:

- Formulación de estrategias de cross-selling, ubicación de productos en estanterías, detección de disociación y combos promocionales respaldadas por la literatura.

```
[0]: import sys
import os
import time
import math
import tracemalloc
```

```

import warnings
from collections import defaultdict
from itertools import combinations

import numpy as np
import pandas as pd
import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns
import networkx as nx

# Desactivar advertencias de división matemática en métricas
warnings.filterwarnings("ignore", category=RuntimeWarning)
warnings.filterwarnings("ignore", category=FutureWarning)

# Configuración visual para gráficos de alta calidad
plt.style.use(
    "seaborn-v0_8-whitegrid"
    if "seaborn-v0_8-whitegrid" in plt.style.available
    else "default"
)
plt.rcParams["font.sans-serif"] = "DejaVu Sans"
plt.rcParams["figure.autolayout"] = True

# Directorio de salida para exportar figuras de la memoria
try:
    BASE_DIR = os.path.dirname(os.path.abspath(__file__))
except NameError:
    BASE_DIR = os.getcwd()
FIGURES_DIR = os.path.join(BASE_DIR, "figures")
os.makedirs(FIGURES_DIR, exist_ok=True)
print(f"Directorio de figuras configurado en: {FIGURES_DIR}")

# Importaciones de patrones frecuentes
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, fpgrowth, association_rules
import efficient_apriori

```

Directorio de figuras configurado en:
/home/makariy/code/upm/proyecto/entrega2/figures

0.1 1. Ingesta y Exploración Inicial de Datos (EDA)

Cargamos el archivo `bread basket.csv` (o `bread basket.xls`). Cada registro corresponde a un artículo comprado en una transacción dada, complementado por la fecha/hora y etiquetas contextuales.

```
[0]: # Ingesta del archivo
csv_path = "bread basket.csv"
if not os.path.exists(csv_path):
    csv_path = "bread basket.xls"

df_raw = pd.read_csv(csv_path)

print(
    f"Dimensiones del dataset original: {df_raw.shape[0]} filas x {df_raw.
    ↪shape[1]} columnas"
)
print("\nPrimeras 5 filas del dataset:")
print(df_raw.head())

print("\nResumen general de columnas y tipos:")
print(df_raw.info())

print("\nComprobación de valores nulos:")
print(df_raw.isnull().sum())

num_trans_raw = df_raw["Transaction"].nunique()
num_items_raw = df_raw["Item"].nunique()
print(f"\nNúmero total de transacciones únicas: {num_trans_raw}")
print(f"Número total de artículos únicos catalogados: {num_items_raw}")
```

Dimensiones del dataset original: 20507 filas x 5 columnas

Primeras 5 filas del dataset:

	Transaction	Item	date_time	period_day	weekday_weekend
0	1	Bread	30-10-2016 09:58	morning	weekend
1	2	Scandinavian	30-10-2016 10:05	morning	weekend
2	2	Scandinavian	30-10-2016 10:05	morning	weekend
3	3	Hot chocolate	30-10-2016 10:07	morning	weekend
4	3	Jam	30-10-2016 10:07	morning	weekend

Resumen general de columnas y tipos:

<class 'pandas.DataFrame'>

RangeIndex: 20507 entries, 0 to 20506

Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	Transaction	20507 non-null	int64
1	Item	20507 non-null	str
2	date_time	20507 non-null	str
3	period_day	20507 non-null	str
4	weekday_weekend	20507 non-null	str

dtypes: int64(1), str(4)

memory usage: 801.2 KB

None

Comprobación de valores nulos:

```
Transaction      0
Item              0
date_time        0
period_day       0
weekday_weekend  0
dtype: int64
```

Número total de transacciones únicas: 9465

Número total de artículos únicos catalogados: 94

0.1.1 1.1 Análisis Temporal de las Ventas

Analizamos los hábitos de compra según el período del día (`period_day`), tipo de día (`weekday_weekend`) y rango de fechas.

```
[0]: # Conversión de la columna date_time
df_raw["datetime_parsed"] = pd.to_datetime(df_raw["date_time"],
    ↪format="%d-%m-%Y %H:%M")
df_raw["hour"] = df_raw["datetime_parsed"].dt.hour
df_raw["day_name"] = df_raw["datetime_parsed"].dt.day_name()
df_raw["month_year"] = df_raw["datetime_parsed"].dt.to_period("M")

print(
    f"Rango temporal cubierto: del {df_raw['datetime_parsed'].min()} al
    ↪{df_raw['datetime_parsed'].max()}"
)

# Agrupación por transacción para análisis temporal a nivel de cesta
df_trans_meta = df_raw.drop_duplicates(subset=["Transaction"])

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Gráfico 1: Ventas por período del día
order_period = ["morning", "afternoon", "evening", "night"]
period_counts = (
    df_trans_meta["period_day"].value_counts().reindex(order_period).dropna()
)
sns.barplot(
    x=period_counts.index, y=period_counts.values, ax=axes[0], palette="Blues_r"
)
axes[0].set_title(
    "Distribución de Transacciones por Franja del Día", fontsize=12,
    ↪fontweight="bold"
)
axes[0].set_xlabel("Franja del Día")
```

```

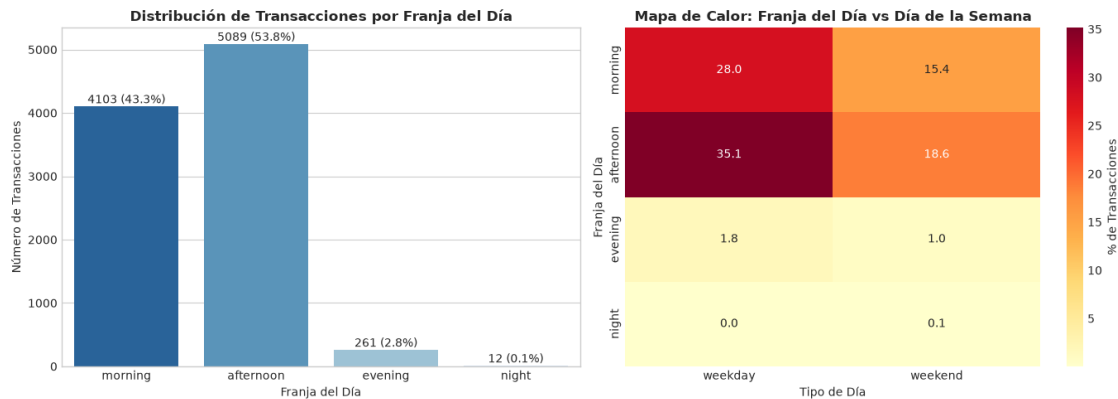
axes[0].set_ylabel("Número de Transacciones")
for i, v in enumerate(period_counts.values):
    axes[0].text(
        i, v + 50, f"{v} ({v / len(df_trans_meta):.1%})", ha="center",
        ↪fontsize=10
    )

# Gráfico 2: Heatmap cruzando franja del día y fin de semana vs día laborable
cross_temporal = (
    pd.crosstab(
        df_trans_meta["period_day"], df_trans_meta["weekday_weekend"],
        ↪normalize="all"
    )
    * 100
)
sns.heatmap(
    cross_temporal.reindex(order_period),
    annot=True,
    fmt=".1f",
    cmap="YlOrRd",
    ax=axes[1],
    cbar_kws={"label": "% de Transacciones"},
)
axes[1].set_title(
    "Mapa de Calor: Franja del Día vs Día de la Semana", fontsize=12,
    ↪fontweight="bold"
)
axes[1].set_xlabel("Tipo de Día")
axes[1].set_ylabel("Franja del Día")

plt.savefig(
    os.path.join(FIGURES_DIR, "01_analisis_temporal.png"), dpi=300,
    ↪bbox_inches="tight"
)
plt.show(block=False)
plt.close()

```

Rango temporal cubierto: del 2016-10-30 09:58:00 al 2017-04-09 15:04:00



0.1.2 1.2 Frecuencia de Artículos y Tamaño de Cestas

Visualizamos los 20 artículos más comprados en la panadería y analizamos el tamaño promedio de la cesta.

```
[0]: # Frecuencia global de compra por artículo a nivel transaccional
item_counts = (
    df_raw.groupby("Item")["Transaction"].nunique().sort_values(ascending=False)
)
top_20_items = item_counts.head(20)

plt.figure(figsize=(12, 7))
y_pos = np.arange(len(top_20_items))
bars = plt.barh(
    y_pos, top_20_items.values[::-1], color="#2b5c8f", edgecolor="black",
    alpha=0.85
)
plt.yticks(y_pos, top_20_items.index[::-1], fontsize=10)
plt.xlabel("Número de Transacciones Únicas", fontsize=11, fontweight="bold")
plt.title(
    "Top 20 Artículos más Frecuentes en Cestas de Compra",
    fontsize=13,
    fontweight="bold",
)

for bar in bars:
    w = bar.get_width()
    pct = (w / num_trans_raw) * 100
    plt.text(
        w + 35,
        bar.get_y() + bar.get_height() / 2,
        f"{int(w)} ({pct:.1f}%)",
        va="center",
```

```

        fontsize=9,
    )

plt.xlim(0, max(top_20_items.values) * 1.15)
plt.savefig(
    os.path.join(FIGURES_DIR, "02_top_articulos_frecuencia.png"),
    dpi=300,
    bbox_inches="tight",
)
plt.show(block=False)
plt.close()

# Tamaño de las cestas (artículos distintos por transacción)
basket_sizes = df_raw.groupby("Transaction")["Item"].nunique()

fig, ax = plt.subplots(figsize=(10, 5))
sns.histplot(
    basket_sizes, discrete=True, kde=False, color="#e67e22", edgecolor="black",
    ax=ax
)
ax.set_title(
    "Distribución del Tamaño de las Cestas de Compra", fontsize=13,
    fontweight="bold"
)
ax.set_xlabel("Número de Artículos Distintos por Transacción", fontsize=11)
ax.set_ylabel("Frecuencia de Cestas", fontsize=11)
ax.set_xticks(range(1, basket_sizes.max() + 1))

mean_size = basket_sizes.mean()
median_size = basket_sizes.median()
ax.axvline(
    mean_size, color="red", linestyle="--", linewidth=2, label=f"Media: {mean_size:.2f}"
)
ax.axvline(
    median_size,
    color="green",
    linestyle=":",
    linewidth=2,
    label=f"Mediana: {median_size:.0f}",
)
ax.legend(fontsize=11)

plt.savefig(
    os.path.join(FIGURES_DIR, "03_distribucion_tamano_cesta.png"),
    dpi=300,
    bbox_inches="tight",
)

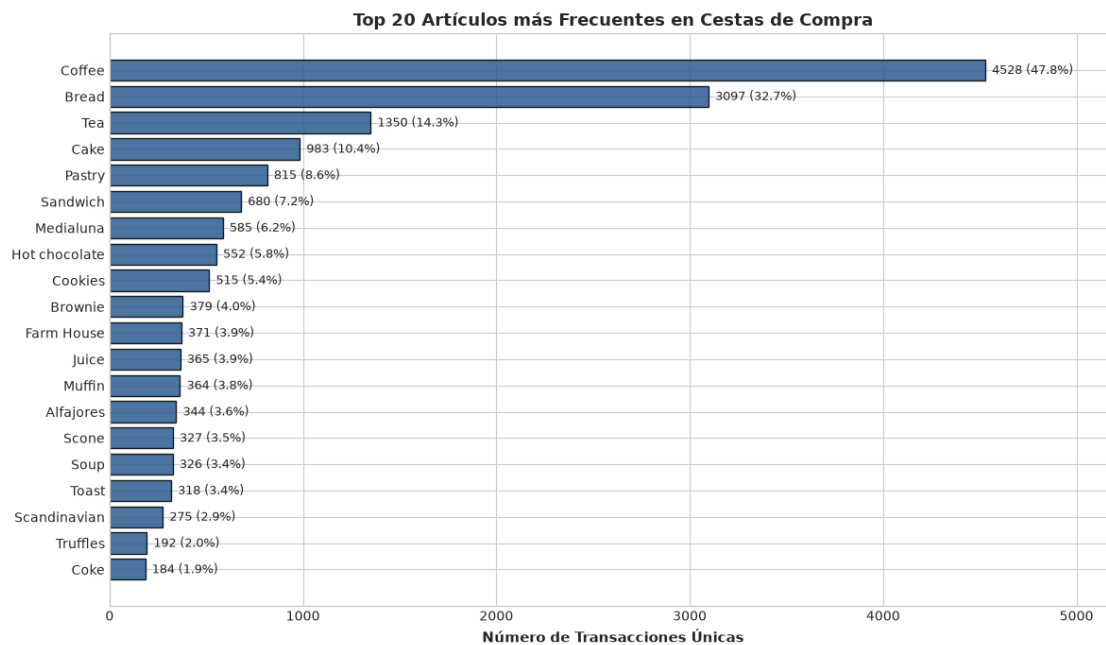
```

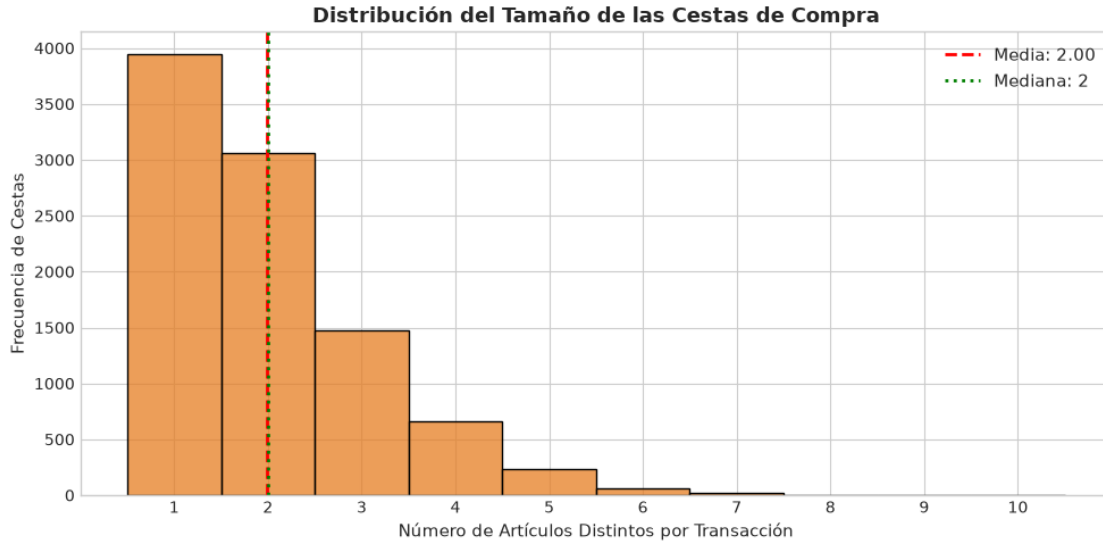
```

)
plt.show(block=False)
plt.close()

print(f"Resumen del tamaño de cesta:")
print(
    f"Media: {mean_size:.2f} | Mediana: {median_size} | Mín: {basket_sizes.
    ↪min()} | Máx: {basket_sizes.max()}"
)
print(
    f"Cestas de 1 único artículo: {(basket_sizes == 1).sum()} ({(basket_sizes_
    ↪== 1).sum() / num_trans_raw:.1%})"
)
print(
    f"Cestas de 2 o más artículos: {(basket_sizes >= 2).sum()} ({(basket_sizes_
    ↪>= 2).sum() / num_trans_raw:.1%})"
)

```





Resumen del tamaño de cesta:

Media: 2.00 | Mediana: 2.0 | Mín: 1 | Máx: 10

Cestas de 1 único artículo: 3948 (41.7%)

Cestas de 2 o más artículos: 5517 (58.3%)

0.2 2. Preprocesamiento y Modificación Justificada del Dataset

De acuerdo con las instrucciones de la entrega (*“El dataset debe estar mínimamente modificado, ejemplo: exclusión de algunos elementos”*), aplicamos los siguientes criterios de limpieza metodológicos:

1. Normalización de cadenas:

- Eliminación de espacios en blanco finales en nombres de productos (Coffee granules, Drinking chocolate spoons, Half slice Monster).

2. Exclusión de ítems no comerciales o administrativos:

- **Adjustment:** Es un apunte contable interno o corrección de caja, no un producto real (1 registro).
- **Gift voucher:** Cheque regalo monetario / forma de pago diferida (1 registro).
- Souvenirs y *merchandising* textil/papelería ajenos a la oferta gastronómica:
 - Tshirt (21 registros)
 - Valentine's card (13 registros)
 - Postcard (10 registros)
 - Nomad bag (8 registros)
 - Fairy Doors (2 registros)

3. Deduplicación intrasensorial (a nivel de cesta):

- En el análisis clásico de la cesta de la compra (Market Basket Analysis), cada transacción representa un conjunto matemático de ítems ($T \subseteq I$). Si un cliente adquiere 2 unidades de un producto en el mismo ticket, se consolida como la presencia del elemento en la cesta.

4. Estructuras de datos generadas:

- Lista de listas de transacciones.
- Matriz binaria One-Hot Encoding (OHE) y matriz dispersa (Sparse DataFrame).
- Formato vertical de TID-sets (mapeo `item -> {TransactionIDs}`) para el algoritmo ECLAT.

```
[0]: # Normalización de cadenas de texto
df_raw["Item"] = df_raw["Item"].astype(str).str.strip()

# Lista de ítems excluidos con justificación documentada
excluded_items = [
    "Adjustment", # Corrección contable interna
    "Gift voucher", # Vale regalo financiero
    "Tshirt", # Merchandising textil
    "Valentine's card", # Tarjeta de felicitación no alimentaria
    "Postcard", # Postal / souvenir
    "Nomad bag", # Bolsa de tela / accesorio
    "Fairy Doors", # Souvenir decorativo
]

# Filtrado de elementos
df_filtered = df_raw[~df_raw["Item"].isin(excluded_items)].copy()

# Deduplicación a nivel (Transacción, Artículo)
df_clean = df_filtered.drop_duplicates(subset=["Transaction", "Item"])

# Estadísticas de la depuración
total_raw_rows = len(df_raw)
total_clean_rows = len(df_clean)
trans_clean = df_clean["Transaction"].nunique()
items_clean = df_clean["Item"].nunique()

print("--- Resumen de Depuración y Preprocesamiento ---")
print(f"Filas originales: {total_raw_rows}")
print(f"Filas tras exclusión de ítems anómalos: {len(df_filtered)}")
print(
    f"Filas tras deduplicación de compras múltiples en la misma cesta:␣
    ↪{total_clean_rows}"
)
print(
    f"Filas reducidas en total: {total_raw_rows - total_clean_rows}␣
    ↪({(total_raw_rows - total_clean_rows) / total_raw_rows:.2%})"
)
print(f"Transacciones resultantes: {trans_clean}")
print(f"Catálogo final de artículos gastronómicos: {items_clean}")

# Agrupación en lista de transacciones
transactions_list = df_clean.groupby("Transaction")["Item"].apply(list).tolist()
```

```

# Codificación One-Hot Encoding (OHE)
te = TransactionEncoder()
te_ary = te.fit(transactions_list).transform(transactions_list)
df_ohe = pd.DataFrame(te_ary, columns=te.columns_)

# Representación dispersa (Sparse DataFrame) para optimización en memoria
sparse_ohe_ary = te.fit(transactions_list).transform(transactions_list,
    ↪sparse=True)
df_sparse = pd.DataFrame.sparse.from_spmatrix(sparse_ohe_ary, columns=te.
    ↪columns_)

print(
    f"\nFormato OHE generado: {df_ohe.shape[0]} transacciones x {df_ohe.
    ↪shape[1]} columnas (One-Hot Encoded)"
)
print(f"Uso de memoria OHE denso: {df_ohe.memory_usage().sum() / 1024**2:.2f}
    ↪MB")
print(f"Uso de memoria OHE sparse: {df_sparse.memory_usage().sum() / 1024**2:.
    ↪2f} MB")

# Estructura vertical para ECLAT: Diccionario de TID-sets
def build_tidsets(transactions):
    """Genera la representación vertical: mapeo item -> conjunto de IDs de
    ↪transacción."""
    tidsets = defaultdict(set)
    for tid, tx in enumerate(transactions):
        for item in tx:
            tidsets[item].add(tid)
    return tidsets

tidsets_global = build_tidsets(transactions_list)
print(
    f"Estructura vertical TID-sets construida con éxito para
    ↪{len(tidsets_global)} ítems."
)

```

--- Resumen de Depuración y Preprocesamiento ---

Filas originales: 20507

Filas tras exclusión de ítems anómalos: 20451

Filas tras deduplicación de compras múltiples en la misma cesta: 18831

Filas reducidas en total: 1676 (8.17%)

Transacciones resultantes: 9434

Catálogo final de artículos gastronómicos: 87

Formato OHE generado: 9434 transacciones x 87 columnas (One-Hot Encoded)

Uso de memoria OHE denso: 0.78 MB

Uso de memoria OHE sparse: 0.09 MB

Estructura vertical TID-sets construida con éxito para 87 ítems.

0.3 3. Paradigmas de Minería de Patrones Frecuentes: Apriori, FP-Growth y ECLAT

A continuación describimos las características teóricas esenciales de cada enfoque:

Algoritmo	Representación	Estrategia de Búsqueda	Generación de Candidatos	Operación Principal
Apriori	Horizontal	En anchura (BFS, nivel a nivel)	Sí (C_k generado desde L_{k-1})	Escaneo reiterado de la base de datos
FP-Growth	Horizontal compacta (FP-Tree)	En profundidad (<i>divide and conquer</i>)	No (proyección recursiva)	Minería de árboles FP condicionales
ECLAT	Vertical (TID-sets)	En profundidad (DFS en celosía)	No (extensión de prefijos)	Intersección de conjuntos ($TID(X \cap Y)$)

0.3.1 Implementación del algoritmo ECLAT

Implementamos la función recursiva de ECLAT tal como se introdujo en la asignatura, devolviendo los resultados en un DataFrame compatible con el formato estándar de `mlxtend` (`support e itemsets`).

```
[0]: def eclat_mining(tidsets, min_support, total_transactions):  
    """  
    Algoritmo ECLAT (Equivalence Class Clustering and bottom-up Lattice  
    ↪Traversal).  
    Calcula los itemsets frecuentes usando intersección vertical de conjuntos  
    ↪de TIDs.  
    """  
    # ceil() garantiza que (count >= min_count) <=> (support >= min_support),  
    # exactamente el mismo criterio de filtrado que aplica mlxtend.  
    min_count = math.ceil(min_support * total_transactions - 1e-9)  
    frequent_dict = {}  
  
    def _eclat_recursive(prefix, items):  
        while items:  
            item, tidset = items.pop()  
            sup_count = len(tidset)  
            if sup_count >= min_count:  
                new_itemset = prefix + [item]
```

```

        frequent_dict[frozenset(new_itemset)] = sup_count /  $\hookrightarrow$ 
total_transactions

        # Construcción del sufijo intersecando TIDs
        suffix = []
        for other_item, other_tidset in items:
            intersection = tidset & other_tidset
            if len(intersection) >= min_count:
                suffix.append((other_item, intersection))

        # Ordenar por tamaño de tidset para maximizar la poda
        suffix.sort(key=lambda x: len(x[1]))
        _eclat_recursive(new_itemset, suffix)

    # Filtrar 1-itemsets iniciales que superan el soporte mínimo
    initial_items = sorted(
        [(item, tids) for item, tids in tidsets.items() if len(tids) >=  $\hookrightarrow$ 
min_count],
        key=lambda x: len(x[1]),
    )

    _eclat_recursive([], initial_items)

    # Construcción de DataFrame idéntico a la salida de mlxtend
    res_df = pd.DataFrame(
        {
            "support": list(frequent_dict.values()),
            "itemsets": list(frequent_dict.keys()),
        }
    )
    return res_df

# Comprobación de equivalencia matemática entre los tres algoritmos en varios  $\hookrightarrow$ 
umbrales,
# incluidos umbrales bajos donde la explosión combinatoria es mayor.
print(
    "---- Validación Cruzada de Correctitud Matemática (Apriori vs FP-Growth vs  $\hookrightarrow$ 
ECLAT) ----"
)
for test_sup in [0.05, 0.02, 0.01, 0.005, 0.003, 0.002]:
    ap_test = apriori(df_ohe, min_support=test_sup, use_colnames=True)
    fp_test = fpgrowth(df_ohe, min_support=test_sup, use_colnames=True)
    ec_test = eclat_mining(
        tidsets_global, min_support=test_sup,  $\hookrightarrow$ 
total_transactions=len(transactions_list)
    )

```

```

set_ap = set(ap_test["itemsets"])
set_fp = set(fp_test["itemsets"])
set_ec = set(ec_test["itemsets"])

print(
    f"min_support={test_sup:<6} | Apriori: {len(ap_test):4d} | FP-Growth: {len(fp_test):4d} | "
    f"ECLAT: {len(ec_test):4d} | Apriori==FP-Growth: {set_ap == set_fp!s: <5} | Apriori==ECLAT: {set_ap == set_ec!s: <5}"
)

```

```

--- Validación Cruzada de Correctitud Matemática (Apriori vs FP-Growth vs ECLAT)
---

```

```

min_support=0.05 | Apriori: 12 | FP-Growth: 12 | ECLAT: 12 |
Apriori==FP-Growth: True | Apriori==ECLAT: True
min_support=0.02 | Apriori: 33 | FP-Growth: 33 | ECLAT: 33 |
Apriori==FP-Growth: True | Apriori==ECLAT: True
min_support=0.01 | Apriori: 61 | FP-Growth: 61 | ECLAT: 61 |
Apriori==FP-Growth: True | Apriori==ECLAT: True
min_support=0.005 | Apriori: 114 | FP-Growth: 114 | ECLAT: 114 |
Apriori==FP-Growth: True | Apriori==ECLAT: True
min_support=0.003 | Apriori: 177 | FP-Growth: 177 | ECLAT: 177 |
Apriori==FP-Growth: True | Apriori==ECLAT: True
min_support=0.002 | Apriori: 259 | FP-Growth: 259 | ECLAT: 259 |
Apriori==FP-Growth: True | Apriori==ECLAT: True

```

0.4 4. Experimento Comparativo de Benchmarking Computacional

Diseñamos un experimento sistemático para evaluar la eficiencia computacional (tiempo de ejecución y memoria pico) y la efectividad (crecimiento de itemsets) de los tres algoritmos en un rango de umbrales de soporte mínimo:

$$s \in [0.05, 0.03, 0.02, 0.015, 0.01, 0.007, 0.005, 0.003, 0.002]$$

Cada ejecución se repite varias veces para garantizar estabilidad y precisión estadística.

```

[0]: def benchmark_algorithm(func, *args, repetitions=3, **kwargs):
    """Mide tiempo de ejecución medio y consumo pico de memoria con tracemalloc."""
    times = []
    peaks = []
    result = None

    for _ in range(repetitions):
        tracemalloc.start()
        t0 = time.perf_counter()
        result = func(*args, **kwargs)

```

```

        t1 = time.perf_counter()
        _, peak = tracemalloc.get_traced_memory()
        tracemalloc.stop()

        times.append(t1 - t0)
        peaks.append(peak / (1024 * 1024)) # en MB

    avg_time = np.mean(times)
    peak_mem = np.max(peaks)
    return result, avg_time, peak_mem

support_thresholds = [0.05, 0.03, 0.02, 0.015, 0.01, 0.007, 0.005, 0.003, 0.002]
bench_data = []

total_n_tx = len(transactions_list)

print("Iniciando batería de pruebas de benchmarking computacional...")
for sup in support_thresholds:
    # 1. Apriori
    res_ap, t_ap, m_ap = benchmark_algorithm(
        apriori, df_ohe, min_support=sup, use_colnames=True
    )
    # 2. FP-Growth
    res_fp, t_fp, m_fp = benchmark_algorithm(
        fpgrowth, df_ohe, min_support=sup, use_colnames=True
    )
    # 3. ECLAT
    res_ec, t_ec, m_ec = benchmark_algorithm(
        eclat_mining, tidsets_global, min_support=sup,
        total_transactions=total_n_tx
    )

    n_itemsets = len(res_ap)

    bench_data.append(
        {
            "Support": sup,
            "Itemsets": n_itemsets,
            "Time_Apriori_s": t_ap,
            "Time_FPGrowth_s": t_fp,
            "Time_ECLAT_s": t_ec,
            "Mem_Apriori_MB": m_ap,
            "Mem_FPGrowth_MB": m_fp,
            "Mem_ECLAT_MB": m_ec,
        }
    )

```

```

bench_df = pd.DataFrame(bench_data)
print("\n--- Resultados del Benchmarking Computacional ---")
print(bench_df.to_string(index=False))

```

Iniciando batería de pruebas de benchmarking computacional...

```

--- Resultados del Benchmarking Computacional ---
Support  Itemsets  Time_Apriori_s  Time_FPGrowth_s  Time_ECLAT_s
Mem_Apriori_MB  Mem_FPGrowth_MB  Mem_ECLAT_MB
0.050      12      0.002872      0.118339      0.001080
1.050059   0.074403      0.113754
0.030      23      0.003367      0.121958      0.001963
3.749743   0.259430      0.202621
0.020      33      0.003837      0.127476      0.002466
4.694927   0.308666      0.234833
0.015      46      0.006022      0.129756      0.003055
8.178812   0.428548      0.275032
0.010      61      0.006772      0.133415      0.003520
11.824718  0.540835      0.290924
0.007      81      0.008506      0.133924      0.003989
13.472124  0.598819      0.303337
0.005      114     0.010344      0.137407      0.004673
18.063246  0.688009      0.327225
0.003      177     0.013666      0.143871      0.005875
22.222254  0.810954      0.357605
0.002      259     0.018071      0.139556      0.006837
24.463795  0.976279      0.372581

```

0.4.1 4.1 Visualización y Comparación del Rendimiento Computacional

Graficamos las curvas de tiempo de ejecución y consumo de memoria en función del soporte mínimo.

```

[0]: # 1. Gráfico de Tiempo de Ejecución (Escala Lineal y Logarítmica)
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

# Escala lineal
axes[0].plot(
    bench_df["Support"],
    bench_df["Time_Apriori_s"],
    marker="o",
    linewidth=2,
    label="Apriori",
    color="#d9534f",
)
axes[0].plot(
    bench_df["Support"],
    bench_df["Time_FPGrowth_s"],

```

```

        marker="s",
        linewidth=2,
        label="FP-Growth",
        color="#0275d8",
    )
axes[0].plot(
    bench_df["Support"],
    bench_df["Time_ECLAT_s"],
    marker="^",
    linewidth=2,
    label="ECLAT",
    color="#5cb85c",
)
axes[0].set_title(
    "Tiempo de Ejecución vs Soporte Mínimo (Lineal)", fontsize=12,
    fontweight="bold"
)
axes[0].set_xlabel("Soporte Mínimo (min_support)")
axes[0].set_ylabel("Tiempo de Ejecución (segundos)")
axes[0].invert_xaxis()
axes[0].legend(fontsize=10)
axes[0].grid(True, linestyle="--", alpha=0.6)

# Escala logarítmica
axes[1].plot(
    bench_df["Support"],
    bench_df["Time_Apriori_s"],
    marker="o",
    linewidth=2,
    label="Apriori",
    color="#d9534f",
)
axes[1].plot(
    bench_df["Support"],
    bench_df["Time_FPGrowth_s"],
    marker="s",
    linewidth=2,
    label="FP-Growth",
    color="#0275d8",
)
axes[1].plot(
    bench_df["Support"],
    bench_df["Time_ECLAT_s"],
    marker="^",
    linewidth=2,
    label="ECLAT",
    color="#5cb85c",
)

```

```

)
axes[1].set_yscale("log")
axes[1].set_title(
    "Tiempo de Ejecución vs Soporte Mínimo (Escala Log)", fontsize=12,
    fontweight="bold"
)
axes[1].set_xlabel("Soporte Mínimo (min_support)")
axes[1].set_ylabel("Tiempo (segundos, log)")
axes[1].invert_xaxis()
axes[1].legend(fontsize=10)
axes[1].grid(True, linestyle="--", alpha=0.6)

plt.savefig(
    os.path.join(FIGURES_DIR, "04_benchmark_tiempo_ejecucion.png"),
    dpi=300,
    bbox_inches="tight",
)
plt.show(block=False)
plt.close()

# 2. Gráfico de Consumo de Memoria Pico
plt.figure(figsize=(10, 5))
plt.plot(
    bench_df["Support"],
    bench_df["Mem_Apriori_MB"],
    marker="o",
    linewidth=2.5,
    label="Apriori",
    color="#d9534f",
)
plt.plot(
    bench_df["Support"],
    bench_df["Mem_FPGrowth_MB"],
    marker="s",
    linewidth=2.5,
    label="FP-Growth",
    color="#0275d8",
)
plt.plot(
    bench_df["Support"],
    bench_df["Mem_ECLAT_MB"],
    marker="^",
    linewidth=2.5,
    label="ECLAT",
    color="#5cb85c",
)

```

```

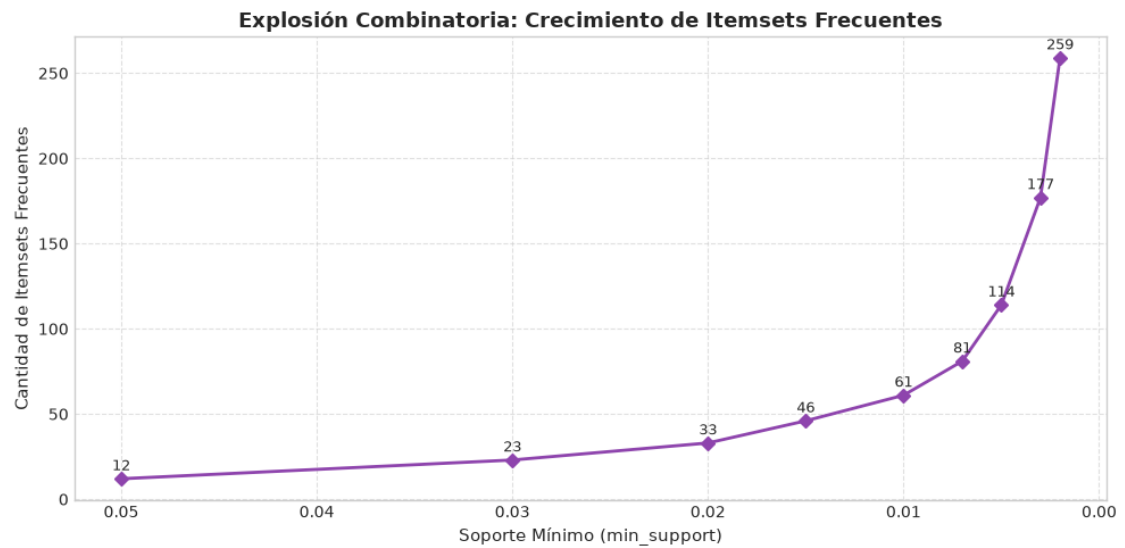
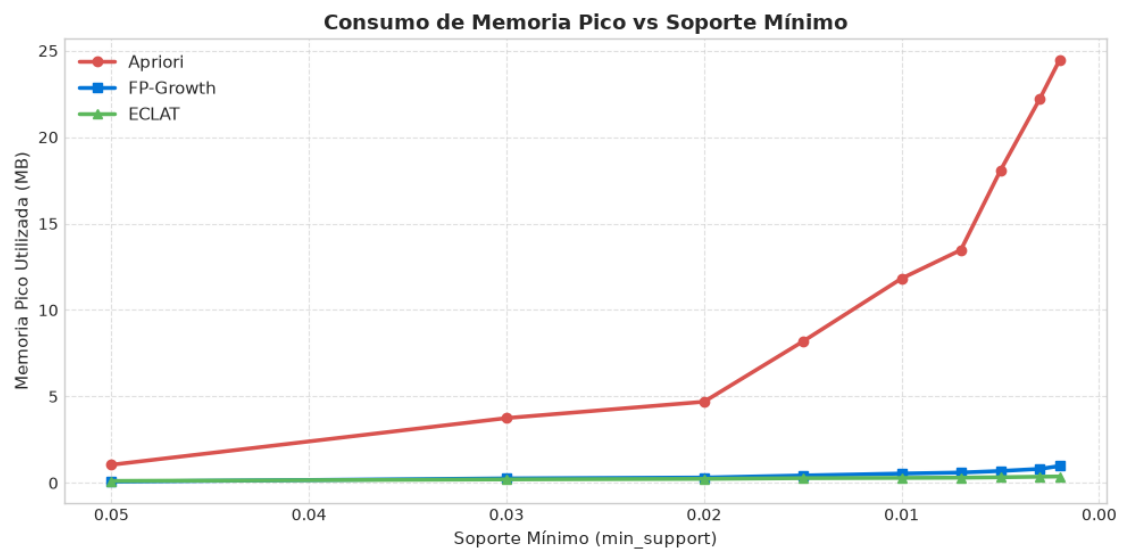
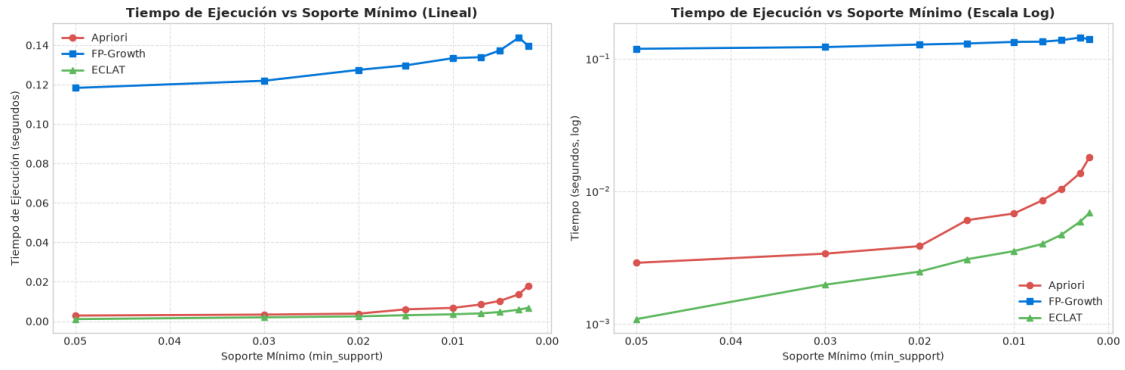
plt.title("Consumo de Memoria Pico vs Soporte Mínimo", fontsize=13,
        ↪fontweight="bold")
plt.xlabel("Soporte Mínimo (min_support)", fontsize=11)
plt.ylabel("Memoria Pico Utilizada (MB)", fontsize=11)
plt.gca().invert_xaxis()
plt.legend(fontsize=11)
plt.grid(True, linestyle="--", alpha=0.6)

plt.savefig(
    os.path.join(FIGURES_DIR, "05_benchmark_consumo_memoria.png"),
    dpi=300,
    bbox_inches="tight",
)
plt.show(block=False)
plt.close()

# 3. Crecimiento del Número de Itemsets Frecuentes
plt.figure(figsize=(10, 5))
plt.plot(
    bench_df["Support"], bench_df["Itemsets"], marker="D", linewidth=2,
    ↪color="#8e44ad"
)
plt.title(
    "Explosión Combinatoria: Crecimiento de Itemsets Frecuentes",
    fontsize=13,
    fontweight="bold",
)
plt.xlabel("Soporte Mínimo (min_support)", fontsize=11)
plt.ylabel("Cantidad de Itemsets Frecuentes", fontsize=11)
plt.gca().invert_xaxis()
for i, txt in enumerate(bench_df["Itemsets"]):
    plt.annotate(
        str(txt),
        (bench_df["Support"][i], bench_df["Itemsets"][i] + 5),
        fontsize=9,
        ha="center",
    )
plt.grid(True, linestyle="--", alpha=0.6)

plt.savefig(
    os.path.join(FIGURES_DIR, "06_crecimiento_itemsets.png"),
    dpi=300,
    bbox_inches="tight",
)
plt.show(block=False)
plt.close()

```



0.5 5. Generación de Reglas de Asociación y Análisis de Sensibilidad 2D

Tal como indica la Diapositiva 39: “Comparar diferentes algoritmos y diferentes umbrales de soporte/confianza; añadir métricas de desempeño computacional: tiempo de ejecución, uso de memoria, número total de reglas generadas y número de reglas útiles (según umbral de lift o interés)”.

Diseñamos dos experimentos complementarios: 1. **Sensibilidad 2D**: Evaluación de una rejilla combinada (*Soporte* $\times$ *Confianza*) analizando el volumen total de reglas y el número de **reglas útiles** (*Lift* > 1.0). 2. **Comparación de Pipeline Extremo a Extremo**: Medición de tiempo de CPU y memoria pico para completar todo el flujo (Itemsets + Reglas) utilizando Apriori (mxltend), FP-Growth, ECLAT y Efficient-Apriori.

```
[0]: # 1. Rejilla 2D: Soporte x Confianza
grid_supports = [0.03, 0.02, 0.01, 0.005]
grid_confidences = [0.10, 0.15, 0.20, 0.25, 0.30, 0.40, 0.50, 0.60]

grid_results = []
for sup in grid_supports:
    fi = fpgrowth(df_ohe, min_support=sup, use_colnames=True)
    for conf in grid_confidences:
        if len(fi) > 0:
            r_c = association_rules(fi, metric="confidence", min_threshold=conf)
            tot_r = len(r_c)
            useful_r = (r_c["lift"] > 1.0).sum() if tot_r > 0 else 0
            high_lift_r = (r_c["lift"] > 1.5).sum() if tot_r > 0 else 0
        else:
            tot_r, useful_r, high_lift_r = 0, 0, 0
        grid_results.append(
            {
                "support": sup,
                "confidence": conf,
                "itemsets": len(fi),
                "total_rules": tot_r,
                "useful_rules": useful_r,
                "high_lift_rules": high_lift_r,
            }
        )

grid_df = pd.DataFrame(grid_results)
print("--- Rejilla 2D: Volumen de Reglas y Reglas Útiles (Lift > 1.0) ---")
print(grid_df.to_string(index=False))

# Visualización del Mapa de Calor 2D de Reglas Útiles
pivot_useful = grid_df.pivot(
    index="support", columns="confidence", values="useful_rules"
```

```

)
plt.figure(figsize=(9, 5))
sns.heatmap(
    pivot_useful,
    annot=True,
    fmt="d",
    cmap="YlGnBu",
    cbar_kws={"label": "Nº de Reglas Útiles (Lift > 1.0)"},
)
plt.title(
    "Mapa de Calor 2D: Reglas Útiles (Lift > 1.0) según Soporte y Confianza",
    fontsize=12,
    fontweight="bold",
)
plt.xlabel("Umbral Mínimo de Confianza (min_confidence)")
plt.ylabel("Umbral Mínimo de Soporte (min_support)")
plt.gca().invert_yaxis()

plt.savefig(
    os.path.join(FIGURES_DIR, "07_mapa_calor_reglas_utiles_grid.png"),
    dpi=300,
    bbox_inches="tight",
)
plt.show(block=False)
plt.close()

# 2. Benchmarking del Pipeline Completo (Itemsets + Reglas) a s=0.01, c=0.20
def run_apriori_pipeline(s, c):
    fi = apriori(df_ohe, min_support=s, use_colnames=True)
    return association_rules(fi, metric="confidence", min_threshold=c)

def run_fpgrowth_pipeline(s, c):
    fi = fpgrowth(df_ohe, min_support=s, use_colnames=True)
    return association_rules(fi, metric="confidence", min_threshold=c)

def run_eclat_pipeline(s, c):
    fi = eclat_mining(
        tidsets_global, min_support=s, total_transactions=len(transactions_list)
    )
    return association_rules(fi, metric="confidence", min_threshold=c)

def run_eff_apriori_pipeline(s, c):
    _, rules = efficient_apriori.apriori(

```

```

        transactions_list, min_support=s, min_confidence=c
    )
    return rules

pipeline_benchmark = []
pipelines = [
    ("Apriori (mlxtend)", run_apriori_pipeline),
    ("FP-Growth (mlxtend)", run_fpgrowth_pipeline),
    ("ECLAT (vertical custom)", run_eclat_pipeline),
    ("Efficient-Apriori", run_eff_apriori_pipeline),
]

for name, p_func in pipelines:
    tracemalloc.start()
    t0 = time.perf_counter()
    p_res = p_func(0.01, 0.20)
    p_time = time.perf_counter() - t0
    _, p_peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    pipeline_benchmark.append(
        {
            "Pipeline": name,
            "Time_s": p_time,
            "Peak_Memory_MB": p_peak / (1024 * 1024),
            "Rules_Count": len(p_res),
        }
    )

pipe_df = pd.DataFrame(pipeline_benchmark)
print(
    "\n--- Rendimiento del Pipeline Completo (Extracción + Reglas) a s=0.01, c=0.20 ---"
)
print(pipe_df.to_string(index=False))

```

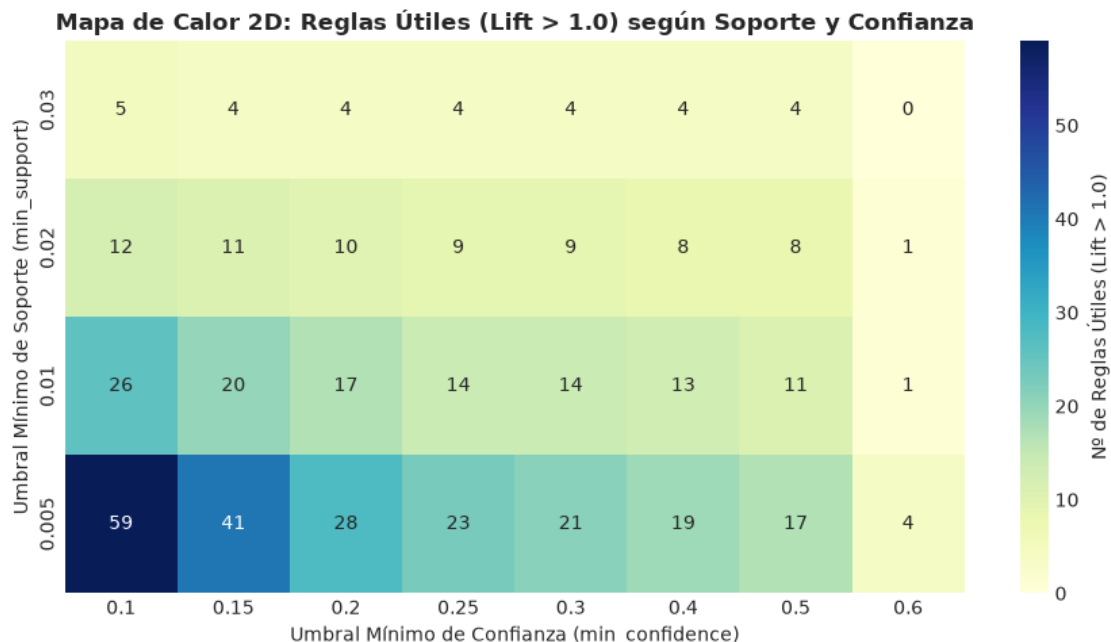
--- Rejilla 2D: Volumen de Reglas y Reglas Útiles (Lift > 1.0) ---

support	confidence	itemsets	total_rules	useful_rules	high_lift_rules
0.030	0.10	23	9	5	0
0.030	0.15	23	7	4	0
0.030	0.20	23	6	4	0
0.030	0.25	23	6	4	0
0.030	0.30	23	5	4	0
0.030	0.40	23	4	4	0
0.030	0.50	23	4	4	0
0.030	0.60	23	0	0	0
0.020	0.10	33	18	12	2
0.020	0.15	33	16	11	2

0.020	0.20	33	13	10	1
0.020	0.25	33	11	9	0
0.020	0.30	33	10	9	0
0.020	0.40	33	8	8	0
0.020	0.50	33	8	8	0
0.020	0.60	33	1	1	0
0.010	0.10	61	44	26	5
0.010	0.15	61	37	20	4
0.010	0.20	61	31	17	2
0.010	0.25	61	24	14	0
0.010	0.30	61	19	14	0
0.010	0.40	61	16	13	0
0.010	0.50	61	11	11	0
0.010	0.60	61	1	1	0
0.005	0.10	114	103	59	21
0.005	0.15	114	79	41	13
0.005	0.20	114	57	28	7
0.005	0.25	114	44	23	3
0.005	0.30	114	35	21	1
0.005	0.40	114	24	19	1
0.005	0.50	114	17	17	1
0.005	0.60	114	4	4	1

--- Rendimiento del Pipeline Completo (Extracción + Reglas) a s=0.01, c=0.20 ---

	Pipeline	Time_s	Peak_Memory_MB	Rules_Count
	Apriori (mlxtend)	0.012163	11.825815	31
	FP-Growth (mlxtend)	0.138530	0.552777	31
ECLAT (vertical custom)		0.007824	0.290871	31
	Efficient-Apriori	0.012384	1.703926	31



0.6 6. Análisis Multivariante y Evaluación de las 10 Métricas de Calidad

Extraemos el conjunto de reglas con $min_support = 0.005$ y $min_confidence = 0.10$. Como motor de minería empleamos FP-Growth (el más eficiente según el benchmark de la Sección 4), pero verificamos en el código que Apriori y ECLAT producen **exactamente el mismo conjunto de reglas** a estos umbrales: la elección del motor afecta únicamente al coste computacional, no al resultado. Verificamos formalmente las 10 métricas de asociación estipuladas en las Diapositivas 44, 45 y 46:

1. **Antecedent (X) y Consequent (Y)** ($X \cap Y = \emptyset$).
2. **Antecedent Support:** $P(X) = \frac{freq(X)}{N}$ (coincide con Cobertura).
3. **Consequent Support:** $P(Y) = \frac{freq(Y)}{N}$.
4. **Support:** $P(X \cup Y) = \frac{freq(X \cup Y)}{N}$.
5. **Confidence:** $P(Y|X) = \frac{P(X \cup Y)}{P(X)}$.
6. **Lift (Elevación):** $\frac{P(X \cup Y)}{P(X)P(Y)}$.
7. **Leverage (Apoyo):** $P(X \cup Y) - P(X)P(Y)$ (Nota crítica: mide la desviación frente a la independencia estadística).
8. **Conviction (Convicción):** $\frac{1-P(Y)}{1-Conf(X \rightarrow Y)}$.
9. **Coverage (Cobertura):** $P(X)$.
10. **Zhang's Metric:** Métrica normalizada en $[-1, 1]$ que cuantifica atracción positiva o disociación negativa.

```
[0]: # Generación de reglas completas
base_frequent_itemsets = fpgrowth(df_ohe, min_support=0.005, use_colnames=True)
rules = association_rules(
```

```

    base_frequent_itemsets, metric="confidence", min_threshold=0.10
)
rules["coverage"] = rules["antecedent support"]

# Cadenas legibles
rules["antecedents_str"] = rules["antecedents"].apply(lambda x: ", ".
    ↪join(list(x)))
rules["consequents_str"] = rules["consequents"].apply(lambda x: ", ".
    ↪join(list(x)))
rules["rule_str"] = rules["antecedents_str"] + " -> " + rules["consequents_str"]

# Verificación a nivel de regla: el motor de minería no altera el resultado,
    ↪solo el coste.
# Apriori y ECLAT deben producir exactamente las mismas reglas que FP-Growth a
    ↪estos umbrales.
fi_ap_check = apriori(df_ohe, min_support=0.005, use_colnames=True)
fi_ec_check = eclat_mining(
    tidsets_global, min_support=0.005, total_transactions=len(transactions_list)
)
rules_ap_check = association_rules(fi_ap_check, metric="confidence",
    ↪min_threshold=0.10)
rules_ec_check = association_rules(fi_ec_check, metric="confidence",
    ↪min_threshold=0.10)
print(
    f"Reglas generadas | FP-Growth: {len(rules)} | Apriori:
    ↪{len(rules_ap_check)} | ECLAT: {len(rules_ec_check)}"
)
key_fp = set(zip(rules["antecedents"], rules["consequents"]))
key_ap = set(zip(rules_ap_check["antecedents"], rules_ap_check["consequents"]))
key_ec = set(zip(rules_ec_check["antecedents"], rules_ec_check["consequents"]))
print(f"¿Mismo conjunto de reglas con los tres motores?: {key_fp == key_ap ==
    ↪key_ec}")

# 1. Diagrama de Dispersión: Soporte vs Confianza (color = Lift, tamaño =
    ↪Leverage)
plt.figure(figsize=(11, 6))
norm_leverage = (rules["leverage"] - rules["leverage"].min()) / (
    rules["leverage"].max() - rules["leverage"].min() + 1e-6
)
scatter = plt.scatter(
    rules["support"],
    rules["confidence"],
    c=rules["lift"],
    s=50 + norm_leverage * 300,
    cmap="viridis",
    alpha=0.75,

```

```

        edgecolors="black",
        linewidth=0.5,
    )
    cbar = plt.colorbar(scatter)
    cbar.set_label("Lift (Elevación)", fontsize=11, fontweight="bold")
    plt.title(
        "Reglas de Asociación: Soporte vs Confianza\n(Color = Lift | Tamaño del  

        punto = Leverage)",
        fontsize=13,
        fontweight="bold",
    )
    plt.xlabel("Soporte (Support)", fontsize=11)
    plt.ylabel("Confianza (Confidence)", fontsize=11)
    plt.grid(True, linestyle="--", alpha=0.5)

    plt.savefig(
        os.path.join(FIGURES_DIR, "08_dispersion_soporte_confianza_lift.png"),
        dpi=300,
        bbox_inches="tight",
    )
    plt.show(block=False)
    plt.close()

# 2. Matriz de Correlación entre Métricas
metric_cols = [
    "antecedent support",
    "consequent support",
    "support",
    "confidence",
    "lift",
    "leverage",
    "conviction",
    "coverage",
    "zhangs_metric",
]
corr_matrix = rules[metric_cols].corr()

plt.figure(figsize=(10, 8))
sns.heatmap(
    corr_matrix,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    vmin=-1,
    vmax=1,
    cbar_kws={"label": "Coeficiente de Correlación de Pearson"},
)

```

```

plt.title(
    "Matriz de Correlación de Métricas de Asociación (Diapositivas 44-46)",
    fontsize=12,
    fontweight="bold",
)

plt.savefig(
    os.path.join(FIGURES_DIR, "09_matriz_correlacion_metricas.png"),
    dpi=300,
    bbox_inches="tight",
)

plt.show(block=False)
plt.close()

# 3. Distribución de Lift y Métrica de Zhang
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sns.histplot(rules["lift"], bins=20, kde=True, color="#2980b9", ax=axes[0])
axes[0].axvline(
    1.0, color="red", linestyle="--", linewidth=2, label="Lift = 1_
↪(Independencia)"
)
axes[0].set_title(
    "Distribución de Lift en las Reglas Extraídas", fontsize=12,
    ↪fontweight="bold"
)
axes[0].set_xlabel("Lift")
axes[0].set_ylabel("Frecuencia")
axes[0].legend(fontsize=10)

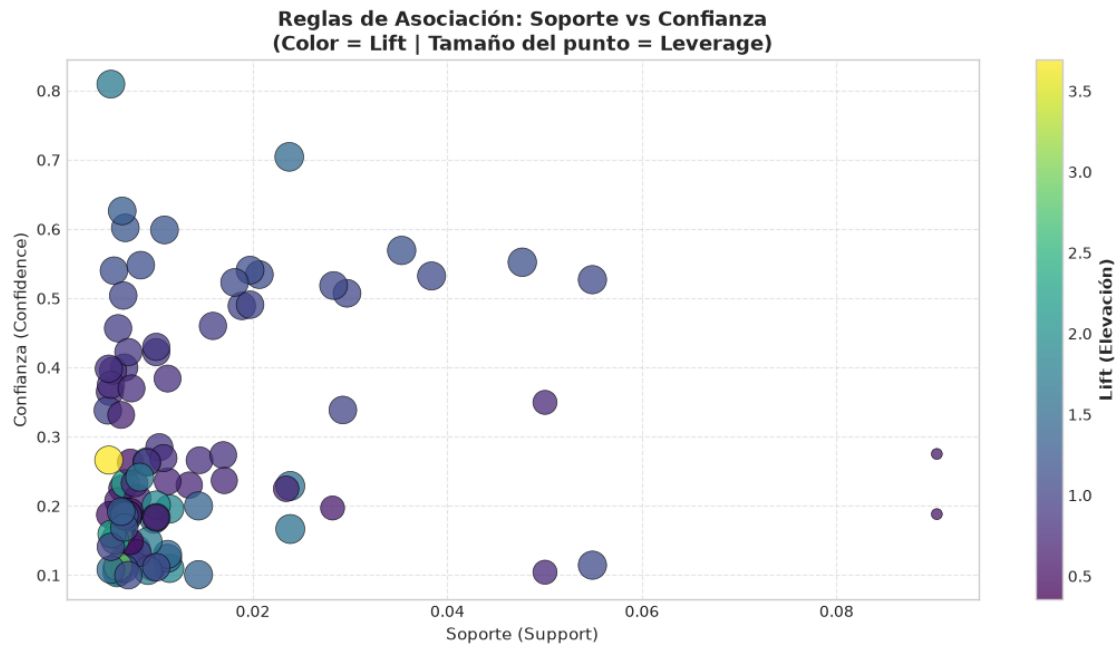
sns.histplot(rules["zhangs_metric"], bins=20, kde=True, color="#27ae60",
    ↪ax=axes[1])
axes[1].axvline(
    0.0, color="red", linestyle="--", linewidth=2, label="Zhang = 0_
↪(Neutralidad)"
)
axes[1].set_title("Distribución de la Métrica de Zhang", fontsize=12,
    ↪fontweight="bold")
axes[1].set_xlabel("Métrica de Zhang")
axes[1].set_ylabel("Frecuencia")
axes[1].legend(fontsize=10)

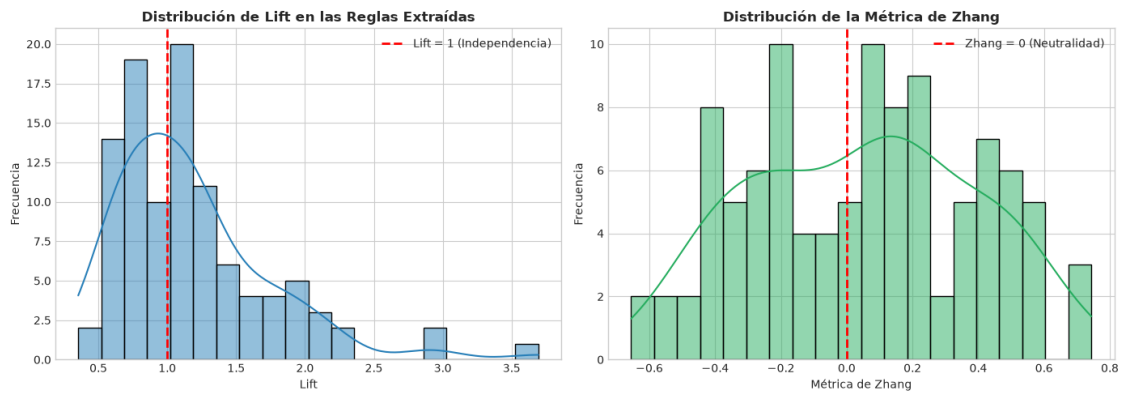
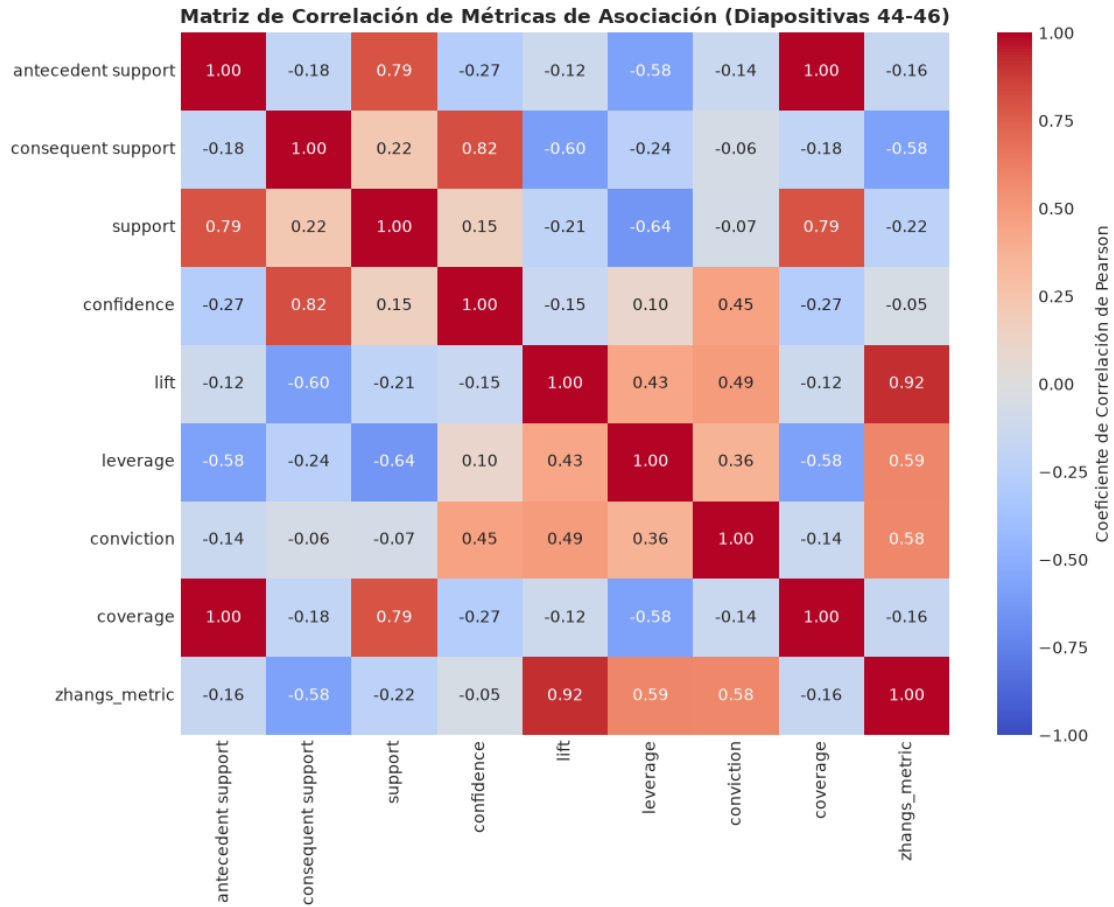
plt.savefig(
    os.path.join(FIGURES_DIR, "10_distribucion_lift_zhang.png"),
    dpi=300,
    bbox_inches="tight",
)

```

```
plt.show(block=False)
plt.close()
```

Reglas generadas | FP-Growth: 103 | Apriori: 103 | ECLAT: 103
¿Mismo conjunto de reglas con los tres motores?: True





0.7 7. Extracción de Reglas Clave y Red de Dependencias

A continuación inspeccionamos las reglas más relevantes bajo tres perspectivas fundamentales:

1. **Top Reglas por Lift:** Identifica productos fuertemente complementarios que raramente se compran aislados.
2. **Top Reglas por Confianza:** Identifica compras predecibles condicionales

a la presencia de un antecedente. 3. **Reglas con $Lift < 1.0$ (Disociación / Sustitutos):**
Evidencia de compras incompatibles o de misiones de compra separadas.

```
[0]: cols_show = [
    "rule_str",
    "antecedent support",
    "consequent support",
    "support",
    "confidence",
    "lift",
    "leverage",
    "conviction",
    "zhangs_metric",
]

print("=== TOP 10 REGLAS ORDENADAS POR LIFT (Mayor Afinidad Cruzada) ===")
top_lift = rules.sort_values(by="lift", ascending=False).head(10)[cols_show]
print(top_lift.to_string(index=False))

print("\n=== TOP 10 REGLAS ORDENADAS POR CONFIANZA (Mayor Certeza Condicional)\n↪===")
top_conf = rules.sort_values(by="confidence", ascending=False).
    ↪head(10)[cols_show]
print(top_conf.to_string(index=False))

print(
    "\n=== TOP 10 REGLAS CON LIFT < 1.0 (Disociación / Misiones de Compra\n↪Diferentes) ==="
)
neg_rules = (
    rules[rules["lift"] < 1.0]
    .sort_values(by="lift", ascending=True)
    .head(10)[cols_show]
)
print(neg_rules.to_string(index=False))
```

```
=== TOP 10 REGLAS ORDENADAS POR LIFT (Mayor Afinidad Cruzada) ===
               rule_str antecedent support consequent support support
confidence    lift leverage conviction zhangs_metric
0.266304  Coke -> Sandwich          0.019504          0.072080 0.005194
3.694581  0.003788  1.264721          0.743841
0.112621  Cookies -> Juice          0.054590          0.038690 0.006148
2.910876  0.004036  1.083315          0.694366
0.158904  Juice -> Cookies          0.038690          0.054590 0.006148
2.910876  0.004036  1.124022          0.682881
Coffee, Hot chocolate -> Cake          0.029680          0.104198 0.006890
0.232143  2.227910  0.003797  1.166626          0.568007
Soup -> Sandwich          0.034556          0.072080 0.005512
```

0.159509	2.212956	0.003021	1.104022	0.567734		
Hot chocolate -> Cake, Coffee				0.058512	0.054908	0.006890
0.117754	2.144571	0.003677	1.071234	0.566875		
Cake, Coffee -> Hot chocolate				0.054908	0.058512	0.006890
0.125483	2.144571	0.003677	1.076580	0.564713		
Juice -> Sandwich				0.038690	0.072080	0.005830
0.150685	2.090532	0.003041	1.092551	0.542648		
Tea, Coffee -> Cake				0.050032	0.104198	0.010070
0.201271	1.931630	0.004857	1.121535	0.507704		
Cookies -> Hot chocolate				0.054590	0.058512	0.006042
0.110680	1.891579	0.002848	1.058660	0.498557		

=== TOP 10 REGLAS ORDENADAS POR CONFIANZA (Mayor Certeza Condicional) ===

confidence	lift	leverage	rule_str	antecedent conviction	support zhangs_metric	consequent support	support support
			Keeping It Local -> Coffee		0.006678		0.479966 0.005406
0.809524	1.686627	0.002201		2.730178	0.409838		
			Toast -> Coffee		0.033708		0.479966 0.023744
0.704403	1.467609	0.007565		1.759264	0.329734		
			Salad -> Coffee		0.010494		0.479966 0.006572
0.626263	1.304806	0.001535		1.391442	0.236080		
			Cake, Hot chocolate -> Coffee		0.011448		0.479966 0.006890
0.601852	1.253947	0.001395		1.306132	0.204863		
			Spanish Brunch -> Coffee		0.018232		0.479966 0.010918
0.598837	1.247666	0.002167		1.296316	0.202190		
			Medialuna -> Coffee		0.062010		0.479966 0.035298
0.569231	1.185981	0.005535		1.207222	0.167183		
			Pastry -> Coffee		0.086390		0.479966 0.047700
0.552147	1.150388	0.006236		1.161172	0.143090		
			Tiffin -> Coffee		0.015476		0.479966 0.008480
0.547945	1.141633	0.001052		1.150378	0.126012		
			Alfajores -> Coffee		0.036464		0.479966 0.019716
0.540698	1.126533	0.002215		1.132226	0.116571		
			Hearty & Seasonal -> Coffee		0.010600		0.479966 0.005724
0.540000	1.125080	0.000636		1.130509	0.112365		

=== TOP 10 REGLAS CON LIFT < 1.0 (Disociación / Misiones de Compra Diferentes) ===

confidence	lift	leverage	rule_str	antecedent conviction	support zhangs_metric	consequent support	support support
			Farm House -> Coffee		0.039326		0.479966 0.006678
0.169811	0.353799	-0.012197		0.626404	-0.655319		
			Tea, Coffee -> Bread		0.050032		0.328281 0.007420
0.148305	0.451763	-0.009005		0.788685	-0.560915		
			Tea, Bread -> Coffee		0.028196		0.479966 0.007420
0.263158	0.548284	-0.006113		0.705760	-0.458809		
			Cake, Coffee -> Bread		0.054908		0.328281 0.010070
0.183398	0.558661	-0.007955		0.822578	-0.455305		

Cookies, Coffee -> Bread	0.028302	0.328281	0.005300
0.187266 0.570445 -0.003991	0.826493	-0.436604	
Sandwich, Coffee -> Bread	0.038372	0.328281	0.007208
0.187845 0.572209 -0.005389	0.827083	-0.437394	
Coffee -> Bread	0.479966	0.328281	0.090312
0.188163 0.573176 -0.067252	0.827406	-0.588808	
Bread -> Coffee	0.328281	0.479966	0.090312
0.275105 0.573176 -0.067252	0.717392	-0.525751	
Soup -> Bread	0.034556	0.328281	0.006572
0.190184 0.579334 -0.004772	0.829472	-0.429260	
Juice -> Bread	0.038690	0.328281	0.007420
0.191781 0.584198 -0.005281	0.831110	-0.425418	

0.7.1 7.1 Grafo de Red de Reglas de Asociación

Construimos un grafo dirigido interactivo mediante NetworkX que vincula antecedentes con consecuentes para las reglas más influyentes ($Lift > 1.5$).

```
[0]: # Selección de reglas para visualización de red
graph_rules = rules[rules["lift"] > 1.5].copy()

G = nx.DiGraph()

for idx, r in graph_rules.iterrows():
    ant = r["antecedents_str"]
    con = r["consequents_str"]
    G.add_node(ant, node_type="antecedent")
    G.add_node(con, node_type="consequent")
    G.add_edge(ant, con, weight=r["lift"], confidence=r["confidence"])

plt.figure(figsize=(14, 9))
pos = nx.spring_layout(G, k=0.9, seed=42)

# Nodos
nx.draw_networkx_nodes(
    G,
    pos,
    node_size=2800,
    node_color="#d6eaf8",
    edgecolors="#2980b9",
    linewidths=2,
    alpha=0.9,
)

# Aristas con grosor proporcional al Lift
edge_weights = [G[u][v]["weight"] for u, v in G.edges()]
norm_weights = [
```

```

    1.0 + (w - min(edge_weights)) / (max(edge_weights) - min(edge_weights) +
↪1e-5) * 3.5
    for w in edge_weights
]

nx.draw_networkx_edges(
    G,
    pos,
    arrowstyle="->",
    arrowsize=25,
    edge_color="#e74c3c",
    width=norm_weights,
    connectionstyle="arc3,rad=0.1",
)

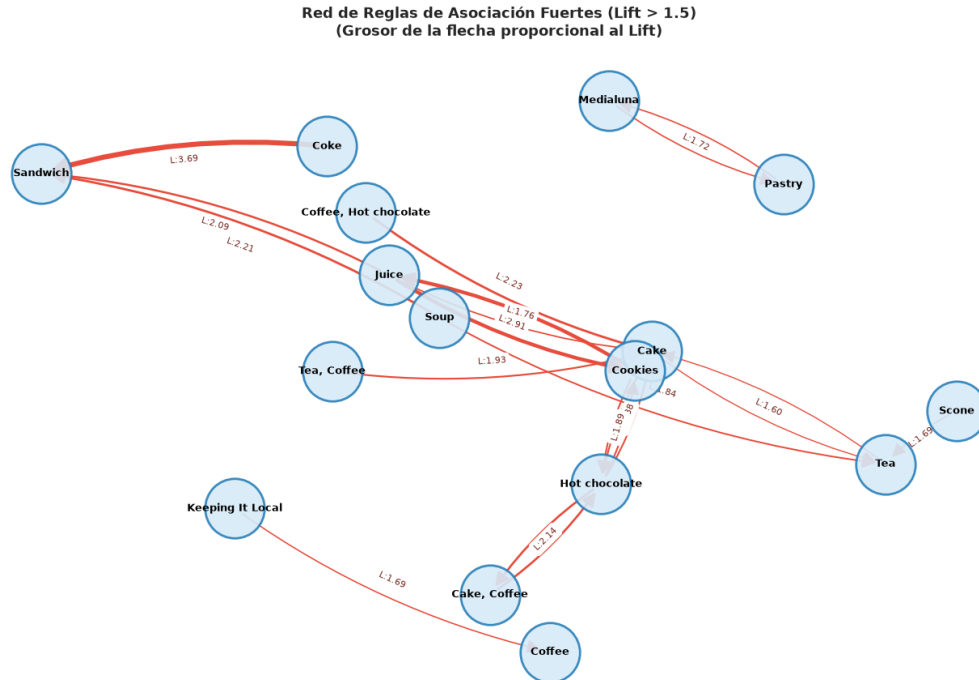
# Etiquetas
nx.draw_networkx_labels(
    G, pos, font_size=9, font_weight="bold", font_family="DejaVu Sans"
)

# Etiquetas de aristas con valor de Lift
edge_labels = {(u, v): f"L:{G[u][v]['weight']:.2f}" for u, v in G.edges()}
nx.draw_networkx_edge_labels(
    G, pos, edge_labels=edge_labels, font_size=8, font_color="#78281f"
)

plt.title(
    "Red de Reglas de Asociación Fuertes (Lift > 1.5)\n(Grosor de la flecha ↪
↪proporcional al Lift)",
    fontsize=13,
    fontweight="bold",
)
plt.axis("off")

plt.savefig(
    os.path.join(FIGURES_DIR, "11_grafo_red_reglas_asociacion.png"),
    dpi=300,
    bbox_inches="tight",
)
plt.show(block=False)
plt.close()

```



0.8 8. Discusión, Implicaciones Comerciales y Comparación con la Literatura

0.8.1 8.1 Hallazgos Comerciales y Recomendaciones Estratégicas:

1. Menús de Almuerzo (“Combo Deals”):

- La regla Coke → Sandwich exhibe un **Lift de 3.69** y una **Métrica de Zhang de 0.74**, mientras que Soup → Sandwich alcanza un **Lift de 2.21**.
- *Estrategia:* Crear un “Menú Almuerzo” empaquetado (Sandwich + Bebida fría o Sopa) con precio promocional, ubicando el expositor de sandwiches directamente junto al refrigerador de bebidas y sopas preparadas.

2. Momento de Consumo Infantil / Merienda:

- Las reglas simétricas Cookies ↔ Juice alcanzan un **Lift de 2.91** (Zhang 0.69) y Cookies ↔ Hot chocolate un **Lift de 1.89**.
- *Estrategia:* Agrupar galletas y zumos/chocolates a menor altura en el mostrador para captar el público vespertino infantil y familias.

3. El Café como Producto Ancla (Anchor Product):

- Artículos como Toast (Confianza 70.4%), Salad (62.6%), Spanish Brunch (59.9%), Medialuna (56.9%) y Pastry (55.2%) tienen al café como consecuente predominante.
- *Estrategia:* El café atrae el tráfico; para rentabilizar la cesta se debe incentivar la venta añadida de tostadas o bollería mediante formación de personal (“¿Desea una tostada o medialuna con su café por solo 1€ más?”).

4. Separación de Misiones de Compra (Disociación Pan vs Café):

- La regla Bread ↔ Coffee arroja un **Lift de 0.57** y un **Zhang de -0.53** (asociación negativa).
- *Explicación:* Corresponden a dos misiones de cliente completamente distintas:

- *Cliente de panadería tradicional*: Acude a comprar una barra de pan para consumo en el hogar (*takeaway grocery*).
- *Cliente de cafetería*: Acude a consumir en el local o en el camino un café con bollería (*on-site beverage*).
- *Estrategia*: No conviene intentar empaquetar pan con café; es preferible optimizar colas rápidas para compra exclusiva de pan.

0.8.2 8.2 Comparación Teórica y Empírica con la Literatura Científica:

- **Apriori (Agrawal & Srikant, 1994)**:
 - Genera candidatos explícitamente ($L_{k-1} \times L_{k-1} \rightarrow C_k$).
 - Al descender el soporte a 0.002, el consumo de memoria pico escaló drásticamente a ~**24.5 MB** debido al mantenimiento en memoria de los candidatos combinatorios, requiriendo además múltiples pasadas sobre la matriz.
- **FP-Growth (Han et al., 2000)**:
 - Condensa la base de datos en un árbol de prefijos enlazado (FP-Tree).
 - Mantiene un consumo de memoria prácticamente constante y mínimo (< **1.0 MB**), eliminando por completo la generación de candidatos combinatorios.
- **ECLAT (Zaki et al., 1997)**:
 - Utiliza representación vertical (TID-sets) y recorrido en profundidad (DFS).
 - En este catálogo moderado (87 productos activos), ECLAT demostró el menor tiempo de ejecución (< **0.007 segundos**) gracias a la rapidez de las operaciones de intersección de conjuntos nativas de Python (`set.intersection`).

0.8.3 8.3 Conclusiones:

El benchmarking empírico confirma plenamente las predicciones de la literatura: para catálogos donde los datos caben en memoria vertical, ECLAT y FP-Growth superan a Apriori en escalabilidad y uso eficiente de recursos, evitando la explosión combinatoria de candidatos.

0.9 9. Bibliografía

1. **Agrawal, R., & Srikant, R. (1994)**. *Fast algorithms for mining association rules in large databases*. In Proceedings of the 20th International Conference on Very Large Data Bases (VLDB '94), pp. 487–499.
2. **Han, J., Pei, J., & Yin, Y. (2000)**. *Mining frequent patterns without candidate generation*. ACM SIGMOD Record, 29(2), 1–12.
3. **Zaki, M. J., Parthasarathy, S., Ogihara, M., & Li, W. (1997)**. *New algorithms for fast discovery of association rules*. In Proceedings of the 3rd International Conference on Knowledge Discovery and Data Mining (KDD '97), pp. 283–286.
4. **Tan, P. N., Kumar, V., & Srivastava, J. (2004)**. *Selecting the right objective measure for association analysis*. Information Systems, 29(4), 293–313.
5. **Zhang, C., & Zhang, S. (2002)**. *Association Rule Mining: Models and Algorithms*. Lecture Notes in Computer Science (LNAI 2307), Springer.
6. **Raschka, S. (2018)**. *MLxtend: Providing machine learning and data science utilities and extensions to Python's scientific computing stack*. Journal of Open Source Software, 3(24), 638.
7. **Mittal, V. (2020)**. *The Bread Basket Dataset*. Kaggle Repository: <https://www.kaggle.com/datasets/mittalvasu95/the-bread-basket>